IDL0

Datasheet

Suspension data logger for motorcycle and bicycle applications.
Hardware, firmware, transport, and signal-processing reference.

SCOPE

IDL0 is a three-IMU, GPS-anchored data logger for suspension and chassis instrumentation. The device captures raw sensor bytes to SD card during a logging session; all signal-processing math lives in the companion app. This document is the engineering contract between the hardware, firmware, and application layers — every field, frame, byte offset, and pipeline stage on the wire is specified here.

1666 Hz	±32 g	3
MAX IMU RATE	ACCEL RANGE	SCHEMA VERSION

LE	200 MB	IDL0
WIRE BYTE ORDER	STORAGE BUDGET	MAGIC

DOCUMENT
IDL0 Datasheet
v1.0 · 2026-05-04

HARDWARE
Seeed XIAO ESP32-C6
3× LSM6DSO32 · MAX-M10S

SOFTWARE
Flutter · Riverpod
Rust · sci-rs · nalgebra

FRONT MATTER

Revision history

This datasheet describes IDL0 binary schema version 3, the WiFi/BLE transport surface as of 2026-05-04, and the Rust signal-processing pipeline shipped with the companion app. Earlier schema versions remain parseable by the app within a two-week transition window after a schema bump; see §10.3.

Rev	Date	Schema	Notes
1.0	2026-05-04	3	Initial public datasheet. Tracks `docs/IDL0_SPEC.md` v1.0.
0.9	2026-04-12	3	Internal draft. Channel registry consolidated; HRM channels (22, 23) added.
0.8	2026-03-20	3	Schema 3 cut over from prototype's schema 1.

FRONT MATTER

Document conventions

CROSS-REFERENCES

Section pointers render as §N or §N.M. They are clickable in the PDF and resolve to the bookmarked section.

CODE VOICE

`Inline code` is monospace. Constants, types, and field names use the same font as the wire — IBM Plex Mono throughout.

COORDINATE FRAME

ISO 8855, right-hand: X forward, Y left, Z up. See §9.

UNITS

Wherever a number appears, its unit appears with it. The Rust API follows scipy naming wherever it can.

BYTE ORDER

All multi-byte integers on the wire are little-endian (ESP32 native). Applies to every schema version.

NUMERIC LITERALS

Prefix 0x for hex; otherwise decimal. Floats include a decimal point even when integer-valued.

RULE This document is forward-looking. It describes **what the system does**, not the path that led there. Recent change history lives in CHANGELOG.md architectural decisions and tradeoffs live in docs/design_rationale.md.

FRONT MATTER

Contents

FRONT MATTER

Figures

PART 1 ORIENTATION

1 SYSTEM PHILOSOPHY

Two hard domains, no overlap.

> **RULE** **Firmware:** during a logging session, raw capture only — sensor bytes to SD card, minimum clock cycles, no filtering, integration, or signal conditioning. Outside a logging session (boot, calibration, transfer, config) the device may compute as those tasks require. The one absolute, in every mode: **analysis DSP — filtering, integration, FFT, statistics — never runs on the device.**

App. All analysis computation. Rust processing layer (sci-rs + nalgebra) called via `flutter_rust_bridge`. Dart for everything else. The device computes calibration *values*; the app *applies* them when processing a log.

File model. Log files (`.idl0`) are immutable after download. All derived work lives in the workspace file (`.idl0w`). Editing a session never mutates the recording.

The architectural consequence of these two domains is shown in Fig. 4 — the firmware is one thin layer, and the app stacks four. The wire between them is defined byte-for-byte in §5. The processing pipeline executed on that data is defined in §19.

1.1 Decision rule

Does it operate on the **physics of the bike** — sensor data, rotation, filtering, integration, frequency analysis? → Rust.
Does it operate on the **app's data structures**, files, or UI state? → Dart.

This line is absolute. The processing crate (`app/rust/`) has no knowledge of Flutter, files, or BLE; the Dart layers never call sci-rs or nalgebra directly. Every Rust function reachable from Dart is annotated with `flutter_rust_bridge::frb` and exposed through the auto-generated bindings.

PART 2 DEVICE & WIRE

2 HARDWARE

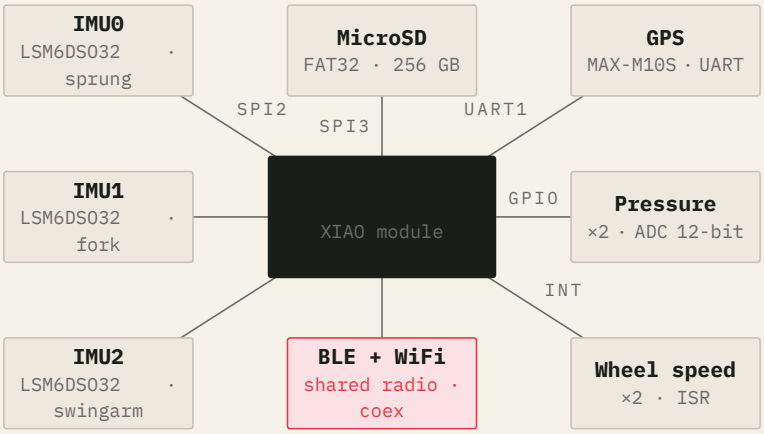


FIG. 1 IDL0 hardware block. The ESP32-C6 fans out to three SPI IMUs, a UART GPS, two ADC pressure inputs, two ISR-counted wheel-pulse inputs, and a single radio shared between BLE (control) and WiFi (data). The radio coexistence rule is enforced in firmware — see §10.4.

2.1 Microcontroller

MODULE	Seeed Studio XIAO ESP32-C6
BUS	SPI2 — shared among IMUs · individual CS per device
FLASH	4 MB · dual-OTA partition layout
WAKE SOURCE	Button · BLE central · battery monitor

2.2 Inertial — LSM6DSO32TR (×3)

Spec	Value
Accel range	±4 / ±8 / ±16 / ±32 g — configurable
Gyro range	±125 / ±250 / ±500 / ±1000 / ±2000 dps
Sample rate	12.5 – 1666 Hz · high-performance mode
Output	16-bit signed · little-endian · raw LSB
Interface	SPI · individual CS

> SPEC **IMU index is a fixed physical location.** IMU0 is the sprung-mass reference on the main PCB; IMU1 is the front unsprung (fork); IMU2 is the rear unsprung (swingarm). Index never enumerates sequentially across boots, and IMU2 is absent on hardtails — the channel registry simply omits its axes (see §5.2).

2.3 GPS — u-blox MAX-M10S

ANTENNA	Linx ANT-GNSSCP-TH25L1 ceramic patch (50 mm ground plane, 50 Ω RF trace)
INTERFACE	UART · 1–10 Hz configurable
ROLE	Absolute time anchor + GPS track for mapping / sectors
NMEA SENTENCES	GGA + RMC default · SBAS enable configurable
DYNAMIC MODEL	automotive (default) · portable / pedestrian / sea / airborne available

2.4 Analog inputs

Two general-purpose ADC channels — PRESSURE_FRONT and PRESSURE_REAR — driven by the ESP32-C6's 12-bit ADC at a maximum 3.3 V input. Primary use is brake pressure transducers, but any 0–3.3 V source is valid. The app applies user-defined scale and offset from the channel registry to convert raw counts to engineering units.

> **WARNING** No input protection on the ADC pins. External sensors must be voltage-divided to ≤ 3.3 V before reaching the device. Out-of-range inputs will damage the SOC.

2.5 Wheel speed

INPUTS	SPEED_FRONT · SPEED_REAR
SENSOR	Hall effect · active-low digital pulse
CADENCE	Interrupt-driven · timestamped per pulse, not polled
ISR	IRAM_ATTR · queued via xQueueSendFromISR
PPR SUPPORT	12 (MTB rotor) · 60 (tone ring) · any user-defined
VELOCITY EQUATION	circ_mm / (ppr × Δt_µs) × 3.6 → km/h

2.6 Device identification

Each ESP32-C6 has a unique 6-byte MAC accessible via esp_efuse_mac_get_default(). The firmware derives two identifiers visible across the transport surface:

Identifier	Source	Format	Visible in
device_id	All 6 MAC bytes	12-char lowercase hex	Binary log header (§5.1), idl0_config.json (§8)
SSID suffix	Last 2 MAC bytes	4-char uppercase hex	WiFi AP SSID IDL0-XXXX (§6)
BLE name suffix	Last 2 MAC bytes	4-char uppercase hex	BLE advertised name IDL0-XXXX (§7)

2.7 Connector — Deutsch DTM15-12PA

Pin	Net	Function
1	+3V3	Power out
2	GND	Ground
3	SPI_SCK	SPI clock
4	MOSI	SPI data out
5	MISO	SPI data in
6	IMU_CS_FRONT	Chip-select · IMU1
7	IMU_CS_REAR	Chip-select · IMU2
8	PRESSURE_FRONT	Analog 0–3.3 V
9	PRESSURE_REAR	Analog 0–3.3 V
10	SPEED_FRONT	Wheel pulse · active-low
11	SPEED_REAR	Wheel pulse · active-low
12	BUTTONS	User input

FIG. 1 12-pin Deutsch DTM connector pinout. All sensor harnesses fan out from this single connector. Pin 1 is keyed.

2.8 PCB and storage

PCB	4-layer · KiCad 9.0 · Seeed Fusion PCBA
STORAGE	MicroSD · FAT32 · 256 GB · SPI3
PEAK WRITE	56 KB/s at max config — 155 MB/hr
BATTERY	1–3 Ah LiPo · JST-PH 2.0 (J3)

3 FIRMWARE

3.1 Core constraint

Zero processing. Read sensor registers. Write raw binary to SD. Nothing else.

3.2 Startup

1. Read `idl0_config.json` from SD root.
2. Parse channel mask, sample rates, enabled sensors.
3. Initialize peripherals.
4. Write session file header.
5. Await start trigger.

3.3 Session triggers

START / STOP	Button press OR BLE CMD_START_LOGGING / CMD_STOP_LOGGING
WIFI ENABLE	On-demand · BLE CMD_WIFI_ON · CMD_WIFI_OFF
BLE STATE	Continuous (control plane)
WIFI ROLE	Data transfer only — file download, config push, OTA

3.4 Record loop

The loop polls sensors at their configured rates and writes raw binary records. GPS records are interspersed as they are received from the UART; wheel pulses produce one record per ISR-debounced event. No value is computed; no value is filtered.

3.5 Sensor failure

If an IMU SPI read fails, firmware writes a zero-filled record with the correct `imu_index` and timestamp. The session continues. The app detects the zero pattern at parse time and surfaces it as a fault region.

3.6 Partition table

Dual-OTA layout on 4 MB flash. The active and pending slots each get 1600 KB. OTA updates stream through `esp_ota_ops.h` into the inactive slot; the bootloader verifies the embedded SHA-256 before the slot swap.

```
nvs,      data, nvs,     0x9000,   24K
phy_init, data, phy,     0xF000,    4K
otadata,  data, ota,     0x10000,   8K
ota_0,    app,  ota_0,   0x20000, 1600K
ota_1,    app,  ota_1,          , 1600K
```

After an OTA-installed image boots, it is in pending-verify state until the app sends `CMD_OTA_CONFIRM` (§7.2). If the device reboots before that confirmation, the bootloader rolls back to the previous slot. See also §6.1.

4 BINARY LOG FORMAT

All multi-byte integers are little-endian (ESP32 native byte order). Applies to every schema version.

4.1 File header

Field	Type	Bytes	Notes
Magic	u8[4]	4	IDL0
Schema version	u8	1	5 3
Session UUID	u8[16]	16	→ 32-char lowercase hex
Device ID	u8[6]	6	→ 12-char lowercase hex
Session start UTC	i64	8	ms, GPS-anchored
Config CRC32	u32	4	CRC-32/ISO-HDLC
IMU channel mask	u32	4	bitfield, see §5.4
IMU count	u8	1	
IMU sample rate	u16	2	Hz
GPS sample rate	u8	1	Hz
Registry count	u8	1	N entries follow
Channel registry	entry[]	N × 40	see §5.2
End marker	u8[4]	4	0xDEADBEEF

FIG. 2 v3 file header — 128 bytes plus a variable-length channel registry. The header carries the session identity, the configuration CRC, and the schema needed to decode the body.

> NOTE Per-sample timing is carried inside the records themselves — IMU and GPS records each carry their own timestamp_us. The header's Session start UTC field stays as the wall-clock anchor; the device's monotonic clock is bound to UTC on first GPS fix.

5.1 Config CRC32 algorithm

Parameter	Value
Polynomial	0x04C11DB7 · reflected 0xEDB88320
Initial value	0xFFFFFFFF
Reflect in / out	yes
Final XOR	0xFFFFFFFF
ESP-IDF call	esp_rom_crc32_le(0, buf, len) — <rom/crc.h>
Dart verifier	package:crclib · Crc32

CRC-32/ISO-HDLC — same family as zlib, PKZIP, and gzip. Computed by the firmware over the raw on-disk JSON bytes of idl0_config.json (§8), exactly as loaded from SD before any whitespace normalisation. This is a corruption / mismatch check only — not security.

5.2 Channel registry entry

Every data source — analog channels, wheel-speed counters, and each individual IMU axis — declares itself in the header. The parser reads the registry once and handles all channel types from a single code path.

Field	Type	Bytes	Notes
channel_id	u8	1	unique per session
data_type	u8	1	0=u8 1=u16 2=u32 3=i8 4=i16 5=i32 6=f32 7=f64
sample_rate_hz	u16	2	0 = event-driven
scale	f32	4	physical = stored × scale + offset
offset	f32	4	
name	u8[20]	20	null-terminated ASCII
units	u8[8]	8	null-terminated ASCII

FIG. 3 40-byte channel registry entry. The entry layout never changes; new sensors arrive by appending new rows.

RULE **Adding a new sensor:** append a registry entry. No other format change. Old app versions see an unknown channel_id, skip those records, and parse everything else normally. The header is forward-compatible; the records are framed.

5.3 IMU channel mask

The mask is interpreted only by the IMU_SAMPLE record decoder. It defines which axes are present in each payload and in what order. Disabled axes have no slot — payloads are variable-stride.

Bits	Channels
0 – 5	IMU0 accel XYZ · gyro XYZ
6 – 11	IMU1 same
12 – 17	IMU2 same
18 – 31	Reserved

5.4 Record types

All records share a common 3-byte framing header:

```
[type:u8][payload_len:u16][payload:N bytes]
```

payload_len is the byte count of the payload only (it does not include the 3-byte header). This enables forward-compatible skipping: on an unknown type, read payload_len and advance.

Tag	Name	Payload
0x01	IMU_SAMPLE	Raw i16 LSB — variable stride per IMU channel mask
0x02	GPS_FIX	Fixed-width parsed GPS fix · 32 bytes
0x03	CHANNEL_SAMPLE	Generic — any channel in the registry
0xFF	SESSION_END	Empty payload · payload_len = 0

SESSION_END semantics. Firmware writes 0xFF and flushes the file when the app sends CMD_STOP_LOGGING (§7.2), when the user presses the stop button, or when battery voltage drops below the soft-cutoff threshold (§10.1). On hard power loss with no SESSION_END, the FAT append guarantees all records up to the last flush survive — the app loads the session normally and surfaces an **interrupted** warning.

5.5 IMU_SAMPLE record (0x01)

Variable stride — only enabled axes are written. The parser computes the payload size from the IMU channel mask once at session load.

Field	Type	Bytes	Present when
imu_index	u8	1	always
timestamp_us	i64	8	always
accel_x	i16	2	mask bit 0/6/12
accel_y	i16	2	mask bit 1/7/13
accel_z	i16	2	mask bit 2/8/14
gyro_x	i16	2	mask bit 3/9/15
gyro_y	i16	2	mask bit 4/10/16
gyro_z	i16	2	mask bit 5/11/17

`timestamp_us` is `esp_timer_get_time()` in microseconds since device boot. Firmware reads the IMU FIFO in bursts and assigns per-sample timestamps walking back from the read instant at the nominal ODR cadence:

> **NOTE** For N samples drained at `t_read`, sample `i` (0 = oldest) is stamped `t_read − (N − 1 − i) × (1_000_000 / ODR)`.

MIN PAYLOAD	9 bytes (imu_index + timestamp_us, no axes enabled)
MAX PAYLOAD	21 bytes (imu_index + timestamp_us + 6 axes)
DROP DETECTION	gap > 1 / ODR between consecutive same-imu_index timestamps

5.6 GPS_FIX record (0x02)

Always 32-byte payload.

Field	Type	Bytes	Notes
gps_epoch_ms	i64	8	UTC ms from the GPS receiver
device_timestamp_us	i64	8	esp_timer_get_time() at fix arrival
latitude	i32	4	deg × 1e7
longitude	i32	4	deg × 1e7
altitude	i16	2	m × 10
speed	u16	2	km/h × 100
heading	u16	2	deg × 100
fix_quality	u8	1	0 = none · 1 = GPS · 2 = DGPS
satellites	u8	1	count

`gps_epoch_ms` and `device_timestamp_us` together anchor the device's monotonic clock to UTC. The file header's `Session start UTC` is set by the firmware on the first valid fix as `gps_epoch_ms − device_timestamp_us / 1000`. Records that precede the first fix carry `gps_epoch_ms = 0` the app uses the first non-zero value to back-fill the anchor for the whole session.

5.7 CHANNEL_SAMPLE record (0x03)

Generic record for all non-IMU, non-GPS channels. Value width is determined by the `data_type` field of the channel's registry entry.

Field	Type	Bytes	Notes
channel_id	u8	1	matches registry entry
timestamp_us	i64	8	µs since boot
value	N	1 – 8	per registry data_type

6 WIFI PROTOCOL

The ESP32 runs as an AP. The phone connects directly — no router.

Parameter	Value
SSID	IDL0-XXXX · XXXX = uppercase hex of MAC bytes 4–5 (see §3.6)
Password	Per-device · current placeholder datalogger123
Device IP	192.168.4.1
Protocol	HTTP/1.1
Coexistence	BLE HRM dropped while WiFi up — see §10.4

6.1 Endpoints

Endpoint	Method	Response	Notes
/files	GET	JSON array	[{"name":"…","size":N}]
/download?file=N	GET	binary stream	Range header supported
/delete?file=N	GET	200 / error	
/config	POST	200 / error	Push idl0_config.json
/ota	POST	200 / error	OTA firmware update

6.2 OTA

The /ota request body is the raw firmware image with Content-Type: application/octet-stream. The device streams the bytes into the inactive OTA partition, then validates the embedded SHA-256 via esp_ota_end().

Result	Meaning
200 · "okn"	Image valid · device reboots after 500 ms
400 · "image validation failed"	SHA-256 mismatch · device keeps running previous image
400 · "short upload"	Content-Length set but fewer bytes received
500 · …	Receive or flash-write failure · device keeps running previous image

The new image boots in pending-verify state. The app commits it with CMD_OTA_CONFIRM (§7.2); without that confirmation, the next reboot rolls back. See also §4.6.

7 BLE PROTOCOL

SCAN — service UUID 000000FF…

ADV — name IDL0-XXXX

CONNECT + MTU exchange

CCCD write — enable Status notify

Notify: WiFi/Logging/Battery/SD/GPS/IMU

Write 0x01 — CMD_WIFI_ON

←Notify: WiFi: ON—

HTTP over WiFi (file transfer)

Write 0x02 — CMD_WIFI_OFF

←Notify: WiFi: OFF—

FIG. 2 BLE handshake. The phone is central; the device is peripheral. Control writes use Write-with-Response — the app waits for the GATT ACK before its call returns.

7.1 GATT

Characteristic	UUID suffix	Type
Service	000000FF-…	—
IMU Data	0000FF01-…	Notify (disabled in v2)
GPS Data	0000FF02-…	Notify (disabled in v2)
Control	0000FF03-…	Write with Response
Status	0000FF04-…	Notify

UUIDs use the long form XXXXXXXX-0000-1000-8000-00805F9B34FB.

7.2 Control commands

Single-byte writes to the Control characteristic.

Byte	Command
0x01	CMD_WIFI_ON
0x02	CMD_WIFI_OFF
0x03	CMD_START_LOGGING
0x04	CMD_STOP_LOGGING
0x05	CMD_CALIBRATE_IMU
0x06	CMD_OTA_CONFIRM

7.3 Status characteristic

UTF-8, newline-delimited, parsed case-insensitively. Unknown lines are silently ignored so the schema may grow without breaking older parsers.

```
WiFi:       ON | OFF
Logging:    RUNNING | STOPPED
Battery:    N%
SD:         OK | FULL | ERROR | ABSENT
GPS:        FIX | NOFIX | ABSENT
IMU:        OK | PARTIAL | ERROR | ABSENT
OTA:        PENDING_VERIFY
HR:         ABSENT | SEARCHING | CONNECTED N | NO_CONTACT N | SUSPENDED
HR_Battery: N%
```

`IMU` is an aggregate across enabled IMUs — `PARTIAL` means at least one is responding and at least one is not. `OTA` is absent in the common case; it appears only between an OTA-installed reboot and `CMD_OTA_CONFIRM`. `HR_Battery` is absent until the first successful battery read on connect; thereafter it persists for the session.

7.4 Central role — Heart Rate Monitor

The firmware runs NimBLE as both **peripheral** (GATT server for the phone) and **central** (GATT client for an HRM strap) on a single radio. Requires `CONFIG_BT_NIMBLE_ROLE_CENTRAL=y` and `CONFIG_BT_NIMBLE_MAX_CONNECTIONS=2`.

The strap exposes the standard Heart Rate Service (`0x180D`) with the Heart Rate Measurement characteristic (`0x2A37`) and the Battery Service (`0x180F`) with the Battery Level characteristic (`0x2A19`). The firmware subscribes to `0x2A37` and reads `0x2A19` once on connect. Each notification produces one `HeartRate` record and N `HR_RR` records, with back-derived timestamps walking from the notification arrival time.

> **WARNING** **WiFi coexistence.** The ESP32-C6 RF coexistence matrix marks SoftAP + BLE as “C1 — unstable”. The firmware drops the HRM connection when WiFi turns on and reconnects when WiFi turns off. HR data is sacrificed during file transfer; file transfer stability is preserved. The phone-facing GATT (peripheral) is unaffected.

8 CONFIGURATION SCHEMA

File: `idl0_config.json` on the SD card root. Pushed manually after review — never pushed automatically.

```
{
  "config_version": 1,
  "device_id": "XXXXXXXXXXXX",
  "bike_profile": { "name": "Trek Session 2024",
                    "default_rider": "Rider Name" },
  "imu": {
    "sample_rate_hz": 833,
    "accel_range_g": 32,
    "gyro_range_dps": 2000,
    "imu0": { "enabled": true,
              "channels": { "accel_x": true, "accel_y": true,
                            "accel_z": true,  "gyro_x":  true,
                            "gyro_y":  true,  "gyro_z":  false } },
    "imu1": { "enabled": true, "accel_range_g": 16, "gyro_range_dps": 500,
              "channels": { ... } },
    "imu2": { "enabled": true, "accel_range_g": 16, "gyro_range_dps": 500,
              "channels": { ... } },
    "orientation": {
      "imu0_rotation_matrix": [[1,0,0],[0,1,0],[0,0,1]],
      "imu1_rotation_matrix": [[1,0,0],[0,1,0],[0,0,1]],
      "imu2_rotation_matrix": [[1,0,0],[0,1,0],[0,0,1]]
    },
    "bias": {
      "imu0": [0,0,0,0,0,0],
      "imu1": [0,0,0,0,0,0],
      "imu2": [0,0,0,0,0,0]
    }
  },
  "gps":   { "sample_rate_hz": 5, "dynamic_model": "automotive",
             "nmea_sentences": ["GGA","RMC"], "sbas_enabled": true },
  "analog":{ "sample_rate_hz": 100, "channels": [] },
  "digital":{ "channels": [] },
  "wheel_speed": {
    "front": { "enabled": false, "points_per_revolution": 12,
               "wheel_circumference_mm": 2300 },
    "rear":  { "enabled": false, "points_per_revolution": 12,
               "wheel_circumference_mm": 2300 }
  },
  "heart_rate_monitor": {
    "enabled": true,
    "device_address": "AA:BB:CC:DD:EE:FF",
    "device_name": "Polar H10 12345678"
  }
}
```

8.1 Valid sample-rate values

The app UI must expose these discrete options. Off-list values produce undefined chip behaviour.

Config path	Valid values	On invalid

imu.sample_rate_hz	High-perf: 12.5 · 26 · 52 · 104 · 208 · 416 · 833 · 1666 Hz Low-power: 1.6 · 12.5 · 26 · 52 · 104 · 208 Hz	Undefined
gps.sample_rate_hz	Integer 1–10 Hz	Clamp or reject
analog.sample_rate_hz	Not yet defined	—

8.2 Read-only fields

The app preserves these on push — they are never user-edited.

DEVICE_ID	12-char lowercase hex of esp_efuse_mac_get_default() (see §3.6)
CONFIG_VERSION	App-managed · increment only for breaking firmware-compat changes

SPEC Firmware ignores unknown JSON fields. The app warns when the device firmware is newer than the app. Breaking changes increment config_version. The push transport is WiFi POST /config — not BLE.

9 COORDINATE SYSTEM

ISO 8855 · right-hand. The vehicle frame the calibration matrix rotates raw sensor vectors into.

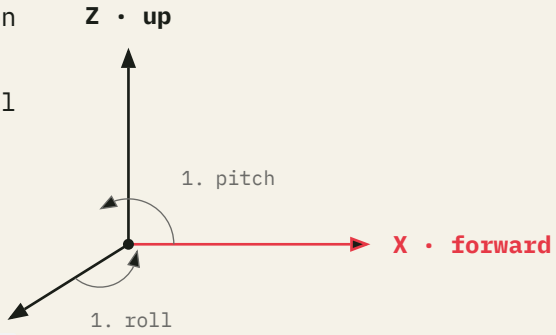
X AXIS	Forward — direction of travel
Y AXIS	Left lateral
Z AXIS	Up

Sign conventions · right-hand rule.

ROLL (ABOUT X)	Positive = left side down
PITCH (ABOUT Y)	Positive = nose up
YAW (ABOUT Z)	Positive = turning left

> NOTE **Suspension note.** Fork compression produces **negative Z** on the sprung IMU and **positive Z** on the unsprung IMU. Account for this sign asymmetry in travel calculation.

Sensor sign convention. The LSM6DSO32 reports specific force reaction, not gravitational acceleration. Stationary and upright, `accel_z ≈ +g`. This is the gravity vector target during calibration (§20).



ISO 8855 · RIGHT-HAND · BIKE-MOUNTED FRA

FIG. 3 Vehicle frame — ISO 8855, right-hand, bike-mounted.

10 DEVICE BEHAVIOR

10.1 Power

SOFT CUTOFF	3.3 V/cell · write SESSION_END · flush SD · BLE notify · power off
HARD CUTOFF	Hardware undervoltage lockout · FAT32 survives ungraceful loss
APP DISPLAY	Battery level shown · warning below 20 %

10.2 SD card

LAYOUT	All session files under /sessions/ at FAT32 root
TEMP FILE	`/sessions/tmp_<boot_ms>.idl0` until first valid GPS fix
FINAL NAME	/sessions/YYYY-MM-DD_HH-MM-SS.idl0 · UTC from gps_epoch_ms
MIN FREE SPACE AT THRESHOLD	200 MB · 1.3 hr at peak load Stop logging · BLE notify · do not crash

10.3 Version compatibility

Surface	Rule
Config JSON	Firmware ignores unknown fields · app warns if firmware newer than app
Binary format	Current schema + immediately prior schema, for 2 weeks after a bump
Binary records	New types skipped by old parsers · never modify existing record layouts

PART 3 **APP**

11 APP ARCHITECTURE

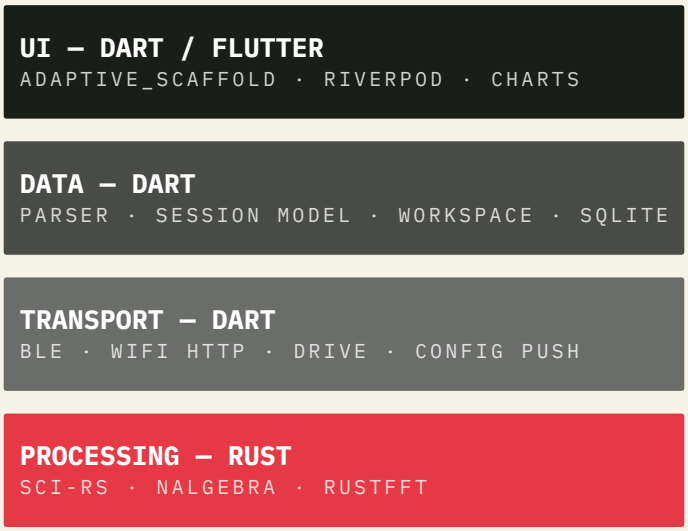


FIG. 4 IDL0 app architecture. Four Dart layers above one Rust processing crate. The Dart–Rust boundary is the flutter_rust_bridge auto-generated bindings.

11.1 Decision rule

Physics of the bike → Rust. App data structures, files, UI → Dart.

11.2 Platform targets

Platform	Capability
Android	Primary — BLE + WiFi + analysis
Desktop (Win / mac)	Analysis · BLE optional
iOS / web	PWA via Flutter web · analysis only · no BLE
Breakpoint	`< 600 px = bottom nav · ≥ 600 px = side rail`

11.3 State management

Riverpod only. No Provider, no Bloc, no `setState` except local widget state. Math channel providers use `FutureProvider.family` — evaluated lazily on demand, never pre-computed.

11.4 File model

File	Ext	Mutable
Raw log	`.idl0`	Never — immutable after download
Workspace	`.idl0w`	Yes — all derived work

`.idl0w` is versioned JSON. It contains lap and sector gates, annotations, math channels, the workbook layout, and channel colours.

11.5 Selection model — XOR

The user is either selecting whole sessions OR individual laps, never mixed. This eliminates the cognitive complexity of comparing “session A as a whole” vs “lap 3 of session B” — typical comparisons are session-vs-session or lap-vs-lap.

Toggling an entry of the inactive kind flips the global mode and clears the inactive set. The Data tab and the Analyze tab’s Session Sheet read and write the same provider.

12 ERROR HANDLING

12.1 Exception hierarchy

```
IdlException
├── ParseException
│   ├── InvalidMagicBytesException
│   ├── UnsupportedSchemaVersionException
│   ├── TruncatedRecordException
│   └── UnknownRecordTypeException
├── CalibrationException
│   └── InsufficientMotionException
├── MathChannelException
│   ├── UnknownChannelException
│   ├── ExpressionSyntaxException
│   └── DivisionByZeroException
└── TransportException
    ├── DeviceNotFoundException
    ├── DeviceUnreachableException
    ├── FileListParseException
    ├── TransferTimeoutException
    └── TransferChecksumException
```

12.2 Behaviour

Exception	Behaviour	User message
`InvalidMagicBytes`	Reject file	“Not a valid IDL0 log”
`UnsupportedSchemaVersion`	Reject file	“Update the app to open this file”
`TruncatedRecord`	Return partial session	“Log incomplete — showing data to [t]”
`UnknownRecordType`	Skip · continue	Silent · debug log
`UnknownChannel`	Inline editor error	“Channel ‘[name]’ not in this session”
`ExpressionSyntax`	Inline editor error	“Syntax error at position N”
`TransferTimeout`	Retry 3× · backoff	“Transfer timed out. Check WiFi.”
`TransferChecksum`	Discard · offer retry	“Transfer error. Retry?”
`CalibrationException`	Abort · keep previous	“Calibration failed — was bike stationary?”

RULE Hard crashes in response to bad data are **never** acceptable. The debug log ring buffer (last 500 entries) is accessible via Settings → tap version 5×.

PART 4 PROCESSING

13 SIGNAL PROCESSING PIPELINE

All steps in Rust (`app/rust/` crate), called via `flutter_rust_bridge`.

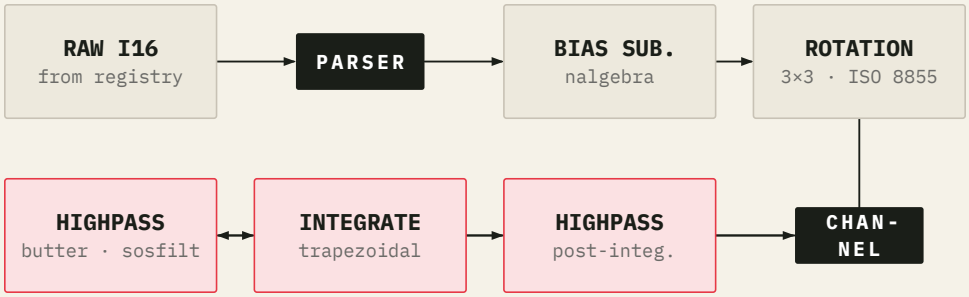


FIG. 5 Default per-IMU-channel pipeline. Steps run in order. Each arrow is one buffer hop. Highpass and integration use sci-rs; rotation uses nalgebra.

13.1 Dependencies

```
sci-rs              = "0.4"   # scipy.signal equivalent
nalgebra            = "0.33"  # linear algebra
rustfft             = "6.2"   # FFT
flutter_rust_bridge = "2"
```

13.2 Default pipeline per IMU channel

1. **Bias subtraction.** `nalgebra` vector subtract using `imu.bias` from config.
2. **Rotation.** `nalgebra` 3×3 matrix multiply → vehicle frame (ISO 8855, see §9).
3. **Input.** IMU samples arrive as physical values (g and dps); the parser applied scale and offset from the channel registry (§5.2) before handoff.
4. **Highpass filter.** sci-rs `butter` order 2, cutoff 0.15–0.3 Hz, applied via `sosfiltfilt`. Rust API: `highpass(data, order, cutoff_hz, sample_rate_hz)`.
5. **Integration · where needed.** Trapezoidal rule. `output[0] = 0.0`, output length = input length. Rust: `integrate(data, sample_rate_hz)`.
6. **Highpass — post-integration.** Second `sosfiltfilt` pass.

All steps user-overridable via math-channel expressions.

13.3 Rust API notes

FILTERS	`highpass(data, order, cutoff_hz, sample_rate_hz)` / `lowpass(…)`
FFT	`fft(data, window)` → one-sided linear magnitude · n/2 + 1 bins
ROTATION MATRIX	Flat 9-element row-major `Vec<f64>` · FRB cannot serialise `[[f64;3];3]`

13.4 Math channel function catalogue

Category	Functions
Filters	`butter(order, cutoff, type, ch)` · `sosfilt(sos, ch)`

Time-domain	`integrate · differentiate · rms · mean · std · median`
Frequency	`fft(ch, window) · spectrogram(ch) · hilbert(ch)`
Correlation	`correlate(a, b) · convolve(ch, kernel)`
Resampling	`resample(ch, hz)`
Math	`abs · sqrt · pow · sign · min · max · clamp · floor · ceil · round`
Trig	`sin · cos · tan · asin · acos · atan · atan2 · sinh · cosh · tanh · deg2rad · rad2deg`
Logic	`if(cond, t, f) · and · or · not`
Range	`ch[t_start:t_end] · ch[lap_n]`
Lap	`current_lap() · lap_start_time(n) · sector_number()`
Variance	`variance_time(ch) · variance_dist(ch)`

WARNING **Logic keyword syntax.** `and`, `or`, `not` are infix / prefix keywords, not call-style functions. Valid: `x > 0 and y < 10`. Invalid: `and(x, y)`. `if` is the only Logic entry that uses call syntax.

13.5 Time as a base channel

`Time` is a synthesised built-in channel with `samples[i] = i / sampleRateHz`, where `sampleRateHz` is the highest non-event-driven channel rate in the session. Not stored on disk — re-synthesised on each session load. Appears in the channel picker alongside `GPS_SpeedKmh` et al., and lets math expressions reference session-relative time directly (the tutorial `LapTime` channel is `Time - lap_start_time(current_lap())`).

14 CALIBRATION

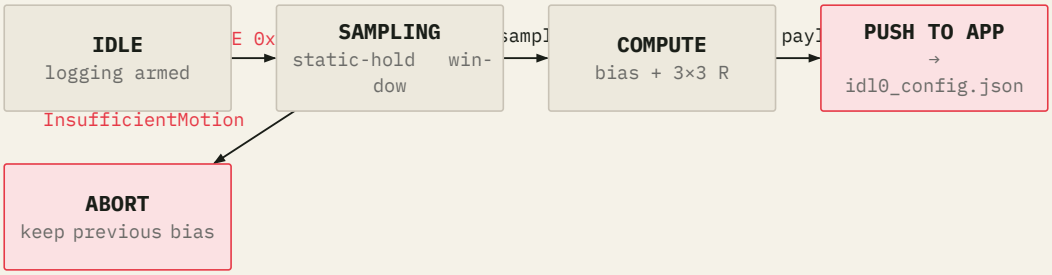


FIG. 6 Calibration state machine. Trigger comes in over BLE as CMD_CALIBRATE_IMU the device samples a static-hold window, computes per-IMU bias and rotation, and ships the result back to the app for persistence in idl0_config.json.

14.1 Trigger and preconditions

TRIGGER	BLE CMD_CALIBRATE_IMU (0x05) · “Calibrate IMUs” button in Device tab
PRECONDITION	Bike stationary · upright · rider off
MODE	Non-logging · self-contained on-device routine

The device captures a static-hold sample window, then computes the per-IMU bias offsets and the 3×3 orientation matrix **on-device**. Computation outside the logging path is permitted (see §1). The device does **not** stream raw IMU data over BLE during calibration.

14.2 Result delivery

The result — per-IMU bias `[ax, ay, az, gx, gy, gz]` and the orientation matrix — is delivered to the app over BLE as a compact, fixed-size payload. The app writes the values into the bike’s `idl0_config.json` (§8 — `bias` / `orientation` fields), so calibration is stored per bike profile and travels with that bike’s config.

> SPEC The firmware never **applies** calibration. It writes raw `i16` LSB values into the log (§5.5). The parser converts raw values to physical units using the per-channel scale and offset from the registry (§5.2). The app then applies bias correction and orientation rotation in the Rust processing layer, matching log to config via the header `config_crc32` (§5.1).

PART A **APPENDIX**

15 CHANNEL REFERENCE

Consolidated reference for every channel ID in the registry at v3 launch. Adding a sensor appends new rows; existing rows never change. The parser applies `physical = stored × scale + offset` using the values stored in the per-session header (§5.2).

ID	Name	Type	Rate (Hz)	Units	Scale
0	IMU0_AccelX	i16	(IMU rate)	g	accel_range_g / 32768
1	IMU0_AccelY	i16	(IMU rate)	g	accel_range_g / 32768
2	IMU0_AccelZ	i16	(IMU rate)	g	accel_range_g / 32768
3	IMU0_GyroX	i16	(IMU rate)	dps	gyro_range_dps / 32768
4	IMU0_GyroY	i16	(IMU rate)	dps	gyro_range_dps / 32768
5	IMU0_GyroZ	i16	(IMU rate)	dps	gyro_range_dps / 32768
6	IMU1_AccelX	i16	(IMU rate)	g	accel_range_g / 32768
…	IMU1 axes	i16	(IMU rate)		as IMU0
11	IMU1_GyroZ	i16	(IMU rate)	dps	as IMU0
12	IMU2_AccelX	i16	(IMU rate)	g	as IMU0
…	IMU2 axes	i16	(IMU rate)		as IMU0
17	IMU2_GyroZ	i16	(IMU rate)	dps	as IMU0
18	WheelFront	u32	0 (event)	pulse	1.0
19	WheelRear	u32	0 (event)	pulse	1.0
20	PressureFront	u16	100	bar	from config
21	PressureRear	u16	100	bar	from config
22	HeartRate	u8	1	bpm	1.0
23	HR_RR	u16	0 (event)	ms	1000/1024

FIG. 4 Channel registry — v3 launch.

The IMU rate column carries “(IMU rate)” because the actual Hz is chosen at session start from `imu.sample_rate_hz` (§8). The registry’s `sample_rate_hz` field carries the resolved value verbatim; the parser does not consult the config.

END OF DOCUMENT · IDL0 DATASHEET v1.0 · 2026-05-04 · idl0.io